

Project Execution Guide

Cloud-Based Research Paper Summarization Platform

Pranjal Mishra

April 15, 2026

- [1. What This Guide Is](#)
- [2. Understanding the Architecture](#)
 - [2.1 Route 53 — DNS](#)
 - [2.2 CloudFront — CDN](#)
 - [2.3 S3 \(two buckets\)](#)
 - [2.4 Cognito — User Accounts](#)
 - [2.5 API Gateway — The Front Door](#)
 - [2.6 Lambda — Serverless Compute](#)
 - [2.7 Step Functions — Workflow Orchestration](#)
 - [2.8 SQS — Message Queue](#)
 - [2.9 DynamoDB — NoSQL Database](#)
 - [2.10 Bedrock — The LLM](#)
 - [2.11 CloudWatch — Logs & Alarms](#)
 - [2.12 IAM — Permissions](#)
- [3. Walking Through a Request](#)
- [4. Build Phases](#)
 - [Phase 0 — Foundation \(Week 3, first half\)](#)
 - [Phase 1 — Walking Skeleton \(Week 3, second half\)](#)
 - [Phase 2 — Search \(Week 4\)](#)
 - [Phase 3 — Summarization \(Weeks 5–6\)](#)
 - [Phase 4 — Polish \(Week 7\)](#)
 - [Phase 5 — Demo Prep \(Week 8\)](#)
- [5. Recommended Repo Structure](#)
- [6. Recommended Tech Stack](#)
- [7. Suggested Division of Labor](#)

- [8. Questions to Settle With the Team](#)
- [9. Mental Mindmap](#)
- [10. Glossary](#)

1. What This Guide Is

This is a practical guide for building the project. Three goals:

1. Explain what each piece of the architecture does and why it's there, so every team member has the same mental model.
2. Lay out the build phases in order, with a clear “done” checkpoint at each step.
3. Give concrete recommendations on tools, repo structure, and division of work.

This is not a contract. Treat it as a starting proposal — pieces are meant to be debated and adjusted with the team.

2. Understanding the Architecture

Each AWS service in our design solves a specific problem. If you understand *why* each one is there, you can defend the architecture in a review and make sensible changes later.

2.1 Route 53 — DNS

Turns `summarizer.example.edu` into an IP address. Without this, users would have to remember something like `d123abc.cloudfront.net`.

2.2 CloudFront — CDN

A content delivery network. Caches our frontend at data centers near end users (faster page loads), adds HTTPS on our domain, and provides basic DDoS protection. Sits in front of both S3 (frontend) and API Gateway (backend).

2.3 S3 (two buckets)

Object storage. We use two buckets for two purposes:

- **Frontend bucket** holds the built Next.js site; served read-only via CloudFront.
- **PDFs bucket** holds raw papers we download. Only our Lambdas touch it. Auto-deletes old files.

2.4 Cognito — User Accounts

A managed user directory. Handles signup, login, password reset, and issues JSON Web Tokens (JWTs) that the frontend sends with every API request. Saves us from writing auth code, which is notoriously easy to get wrong.

Key idea. When a user logs in, Cognito gives them a token. Every subsequent request carries this token. API Gateway validates the token before letting the request through to our code. So our Lambdas can trust that requests are already authenticated.

2.5 API Gateway — The Front Door

A single HTTP entrypoint that routes requests to the right backend handler. Also validates authentication, applies rate limits, and logs everything.

2.6 Lambda — Serverless Compute

Event-driven code execution. Each Lambda is a small function that runs in response to something — an HTTP request, a queue message, a Step Functions state. No servers to manage; we pay per invocation.

Catches. A single Lambda can't run longer than 15 minutes. First invocation after inactivity takes a bit longer ("cold start"). For long-running workflows, we compose multiple Lambdas with Step Functions.

2.7 Step Functions — Workflow Orchestration

A JSON-defined state machine that chains Lambdas together. Our summarization pipeline has five steps, some running in parallel. Step Functions handles retries, error catching, and branching between steps. It also gives us a visual execution history, which is invaluable when debugging.

2.8 SQS — Message Queue

A buffer between "someone wants a summary" and "the pipeline runs." When a user submits a summary request, we drop a message on the queue and return immediately. The pipeline picks up the message asynchronously. This keeps the API responsive even when summarization takes 30+ seconds.

Dead Letter Queue. If a message fails repeatedly (say three times), it lands in a DLQ instead of looping forever. Without this, bad messages cause infinite retries.

2.9 DynamoDB — NoSQL Database

Our primary data store. We use a “single-table design”: one table holds users, jobs, and summaries, distinguished by a key prefix. This is counter-intuitive if you’re coming from SQL, but it’s the canonical DynamoDB pattern and it’s very fast and cheap.

2.10 Bedrock — The LLM

Managed API for Claude Sonnet. Our Lambdas invoke it with IAM permissions — no API key to manage. Same model quality as the direct Anthropic API.

2.11 CloudWatch — Logs & Alarms

Where all logs, metrics, and alarms live. We write structured JSON from every Lambda and wire alarms to fire when things go wrong (errors spike, DLQ fills up, etc.).

2.12 IAM — Permissions

Every cross-service call in AWS is governed by IAM. Each Lambda has its own role with only the permissions it actually needs (least privilege). `searchFn` can’t touch the PDFs bucket; `fetchPDF` can’t read user passwords.

3. Walking Through a Request

The fastest way to cement the mental model: trace what happens when a user summarizes a paper.

1. User hits `summarizer.example.edu`. Route 53 resolves it, CloudFront serves the Next.js frontend from S3.
2. User logs in. Cognito returns a token, stored in the browser.
3. User searches “transformer models in healthcare.” Frontend calls `POST /search` with the token. API Gateway validates the token, invokes `searchFn`, which calls arXiv and returns results.
4. User clicks “Summarize” on a paper. Frontend calls `POST /summarize`. `submitJobFn` writes a `JOB#` record to DynamoDB, puts a message on SQS, and returns `202 Accepted` with a job ID.
5. SQS triggers Step Functions. The state machine runs:
 - `FetchPDF` downloads the paper to S3
 - `ExtractText` converts PDF to plain text
 - `Chunk` splits the text

- MapSummarize calls Bedrock on each chunk in parallel
- ReduceSummary combines the chunks and writes the final summary to DynamoDB

6. Meanwhile, the frontend polls `GET /summaries/{jobId}`. When the job is `DONE`, the summary is displayed.

Every service exists because one of these steps needs it. Remove any one and something breaks.

4. Build Phases

Five phases, sequential. Each has an explicit “done when” checkpoint so we can track progress cleanly.

Phase 0 — Foundation (Week 3, first half)

Goal. Empty but deployable infrastructure.

- Decide on AWS account strategy (class credits vs personal vs shared). Document it.
- Create IAM users for each teammate with developer-level access.
- Set a \$50 AWS Budgets alert.
- Create a single GitHub repo; agree on branch strategy.
- Scaffold the AWS CDK project.
- Set up GitHub Actions to run `cdk synth` on every PR.

Done when. A PR triggers the CI check. Everyone can run `cdk synth` locally.

Phase 1 — Walking Skeleton (Week 3, second half)

Goal. An end-to-end system that does nothing useful, but exercises every layer.

- Deploy the frontend (empty Next.js page) to S3 + CloudFront.
- Deploy Cognito user pool with email signup.
- Deploy API Gateway with one stub Lambda: `GET /health` returning `{status: "ok"}`.
- Frontend has a login screen and a button that calls `/health`.

Done when. A new user can sign up in the deployed app, log in, click a button, and see `{"status": "ok"}` come back.

Why this phase is critical. We solve all the annoying integration problems — IAM permissions, CORS, token flow — before we write any real feature code. After this phase, adding features is fast.

Phase 2 — Search (Week 4)

Goal. Users can search and browse papers.

- searchFn Lambda that queries arXiv.
- Frontend search page with results list.
- Basic empty-state and error handling.

Done when. A user can type a topic and see a list of real papers from arXiv.

Phase 3 — Summarization (Weeks 5–6)

Goal. End-to-end summarization.

- DynamoDB table with the single-table schema.
- submitJobFn and getSummaryFn Lambdas.
- Step Functions state machine with the five pipeline Lambdas.
- Bedrock access confirmed; test with a real paper.
- DLQ configured; CloudWatch alarm on DLQ depth.

Done when. A user submits a paper and gets back a structured summary in under 90 seconds (for papers up to 30 pages).

Phase 4 — Polish (Week 7)

Goal. Production-quality experience.

- Nice UI: loading states, error messages, responsive layout.
- Structured logs everywhere; dashboard with key metrics.
- Load test with ~50 concurrent searches and ~10 concurrent summaries. Fix anything that breaks.
- A real README.

Done when. We can demo to someone outside the class and nothing embarrassing happens.

Phase 5 — Demo Prep (Week 8)

Goal. Smooth final presentation.

- Rehearsed demo script with a known-good paper.
- Slide deck.
- Backup plan in case something goes wrong live.

5. Recommended Repo Structure

```
research-summarizer/
├── README.md
├── .github/
│   └── workflows/ci.yml
├── infra/                                # AWS CDK (all infrastructure as code)
│   ├── bin/app.ts
│   ├── lib/
│   │   ├── frontend-stack.ts
│   │   ├── api-stack.ts
│   │   └── pipeline-stack.ts
│   └── package.json
├── apps/
│   ├── web/                             # Next.js frontend
│   │   ├── app/
│   │   ├── components/
│   │   └── package.json
│   └── api/                              # Lambda handler code
│       ├── handlers/
│       │   ├── search.ts
│       │   ├── submit-job.ts
│       │   ├── get-summary.ts
│       │   └── pipeline/
│       │       ├── fetch-pdf.ts
│       │       ├── extract-text.ts
│       │       ├── chunk.ts
│       │       ├── map-summarize.ts
│       │       └── reduce-summary.ts
│       └── package.json
└── packages/
    └── shared/                          # TypeScript types shared between web and
api
```

Using a **monorepo** (single repo, multiple packages) rather than separate repos because:

- Types defined in `packages/shared` stay in sync between frontend and backend automatically.
- A single PR can change infrastructure, backend, and frontend together.
- CI is simpler to set up.
- For a four-person team, multiple repos create more friction than they save.

6. Recommended Tech Stack

Purpose	Choice	Why
Language everywhere	TypeScript	Typed, shared across frontend and backend
Frontend framework	Next.js 14	Good defaults, easy static export
Infrastructure	AWS CDK v2	Typed IaC; official AWS tool
Lambda runtime	Node.js 20 (LTS)	Current Lambda default
Package manager	pnpm	Good monorepo support
LLM	Claude Sonnet via Bedrock	Quality + AWS-native
Testing	Vitest	Fast, TypeScript-first
CI/CD	GitHub Actions	Free for public repos

7. Suggested Division of Labor

Suggested ownership — everyone reviews each other’s work, but one person “owns” each area end to end.

Area	Owner	Scope
Frontend	Member A	Next.js app, UI, frontend stack
API & Auth	Member B	API Gateway, Cognito, handler Lambdas
Pipeline	Member C	Step Functions, pipeline Lambdas, Bedrock
Data & Ops	Member D	DynamoDB schema, CloudWatch, alarms, DLQ

8. Questions to Settle With the Team

Before we start coding, let’s agree on these:

1. Which AWS account do we use? Class credits, shared team, or individual accounts?
2. Is Bedrock enabled in our account and region? (If not, who requests it?)

3. What's the monthly budget ceiling?
4. How often are we syncing — weekly? Twice a week?
5. How many approvals before a PR merges?
6. Branch strategy: trunk-based (short-lived branches, merge to main) or GitFlow? (Trunk-based is probably right for four people.)
7. Where do non-code notes live? (Recommend: a docs/ folder in the repo.)
8. How do we track tasks? (Recommend: GitHub Issues + Projects.)
9. Slack, Discord, or group chat for daily communication?
10. Who's on point for the final demo logistics?

9. Mental Mindmap

For the presentation and for quick reference, the whole architecture as a verbal decision tree:

- **User visits the site** → DNS resolves → CloudFront serves content
 - **For the UI** → CloudFront returns cached Next.js from S3
 - **For the API** → CloudFront forwards to API Gateway
- **API Gateway receives a request** → Cognito Authorizer validates JWT
 - **Search** → searchFn → arXiv API → response
 - **Submit summary** → submitJobFn → DynamoDB (record job) + SQS (enqueue) → 202
 - **Get summary** → getSummaryFn → DynamoDB → response
- **SQS triggers Step Functions**
 - FetchPDF → S3
 - ExtractText (reads S3)
 - Chunk
 - MapSummarize (parallel) → Bedrock
 - ReduceSummary → DynamoDB
 - **Failure after 3 retries** → DLQ → CloudWatch alarm → SNS email
- **Always running quietly**
 - CloudWatch collects logs and metrics
 - IAM enforces permissions
 - AWS Budgets watches spending

10. Glossary

Term	Meaning
CDK	Cloud Development Kit — AWS's Infrastructure-as-Code tool
CDN	Content Delivery Network
DLQ	Dead Letter Queue
GSI	Global Secondary Index (DynamoDB)
IAM	Identity and Access Management
JWT	JSON Web Token
LLM	Large Language Model
OAI	Origin Access Identity (CloudFront  private S3)
SLO	Service Level Objective
TLS	Transport Layer Security